Injection de dépendances avec Spring (cours 2)

I. Mougenot (isabelle.mougenot@umontpellier.fr)

Faculté des Sciences Université Montpellier

2026

Plan du cours

Focus sur les niveaux domaine et métier

1. Inversion de Contrôle (IoC) et Inversion de Dépendance (DIP)
2. SOLID et les bonnes pratiques de conception
3. Pattern "Service Locator"
4. Pattern d'injection de dépendance (DI)
5. Les services Spring et la logique métier
6. Cas d'usage et exemples de code

UNIVERSITÉ DE MONTPELLIER

Principe de l'inversion de contrôle (IoC)

Notion générale qui traduit que le flux d'exécution est sous le contrôle d'un code tierce partie (dans notre contexte le framework) et non plus sous le contrôle du développeur/code/application elle même. Le contrôle est donc extérieur pour garantir flexibilité et modularité du code (bonnes pratiques de conception)

1. Il est à noter que IoC est lié au principe d'inversion de dépendance (décrit par le D dans l'acronyme SOLID pour de bonnes pratiques)
2. le pattern d'injection de dépendance (DI) permet de diminuer le couplage entre objets (en évitant par exemple que des objets aient la création d'objets sous leur responsabilité)

UNIVERSITÉ DE MONTPELLIER

IoC, DIP et DI

Des principes repris au travers de patterns aboutissant à des solutions concrètes au sein de frameworks

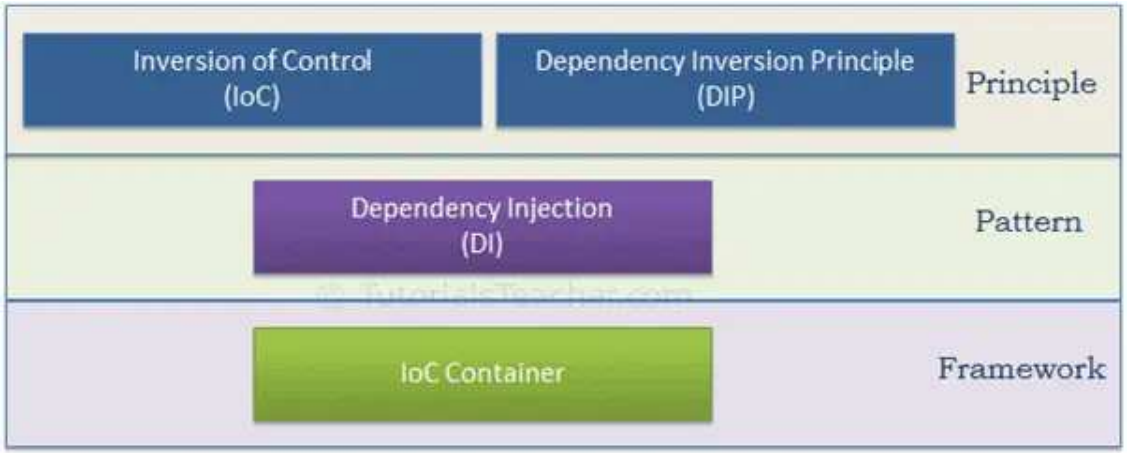

Figure: Contexte IoC

Crédit Image : voir `https://medium.com/@alemarr/`

SOLID et bonnes pratiques de conception objet

Privilégier un code facile à maintenir, tester et faire évoluer

1. **S**ingle Responsibility Principle : un module ne devrait pas avoir plus d'une raison de changer (objectif : en augmenter la cohésion)
2. **O**pen Closed Principle : un module doit pouvoir être étendu sans pouvoir être modifié (objectif : éviter la régression de code).
3. **L**iskov Substitution Principle : les sous-classes doivent pouvoir hériter et implémenter toutes les méthodes et propriétés de la superclasse.
4. **I**nterface Segregation Principle : La dépendance d'une classe à une autre devrait être restreinte à l'interface la plus petite possible.
5. **D**ependency Inversion Principle : les classes ne doivent pas dépendre d'autres classes, mais d'interfaces ou de classes abstraites.

UNIVERSITÉ DE MONTPELLIER

Principe d'inversion de dépendance (DIP)

La relation qui fait que les modules de haut niveau (aspect métier) sont dépendants des modules de bas niveau (aspect implémentation), est inversée afin de rendre les premiers indépendants des seconds. L'idée générale est de miser sur des abstractions

Problèmes posés par trop de dépendances :

1. Rigidité : Les modules fonctionnels sont impactés par les changements des modules techniques
2. Immobilisme : Les modules génériques ne peuvent pas être réutilisés car trop liés à une implémentation particulière.

UNIVERSITÉ DE MONTPELLIER

Illustration partielle SOLID

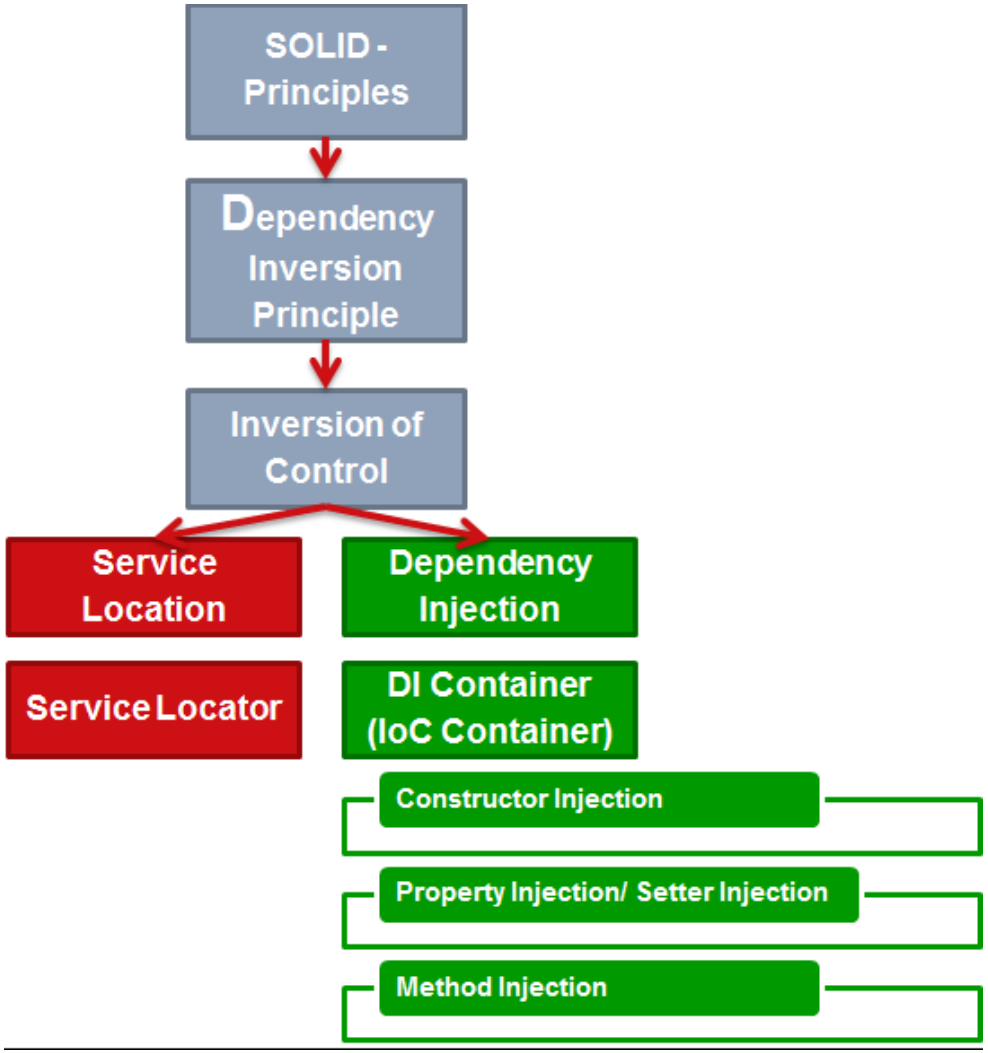

Figure: DIP et deux patterns qui y sont associés

Illustration de la responsabilités entre objets

```
class EmailService {
private SmtpServer smtpServer;
public EmailService()
{ this.smtpServer = new SmtpServer("smtp.example.net", 587); }

public void sendEmail(String to, String subject, String body)
{ // Code mobilisant la variable d'instance smtpServer } }
```

Listing 1: une classe service

Dépendance forte

SmtpServer n'est pas une abstraction ou une interface puisque EmailService spécifie le type concret de l'attribut smtpServer. La création de l'objet SmtpServer nécessite de passer des paramètres

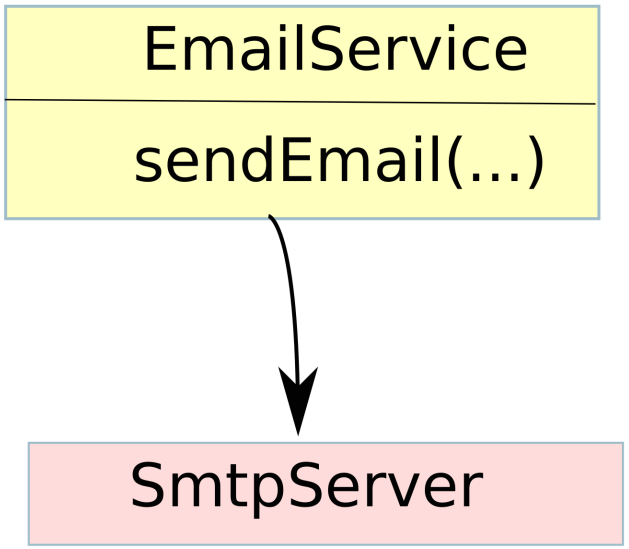

Figure: Dépendances entre classes

Principe DIP

Inversion de la dépendance : les modules de bas niveau doit être conformes aux interfaces définies par des modules de haut niveau

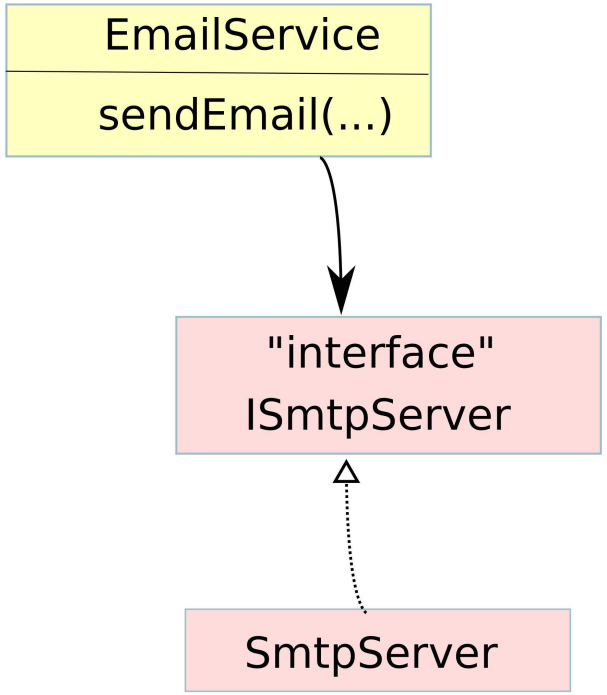

Figure: Mobiliser des interfaces

Principe du "service locator" conçu comme un singleton

```
public class EmailService implements Emailing {
public void sendEmail(String to, String subject, String body)
{ // Code mobilisant une instance smtpServer
SmtpServer smtpServer = SmtpServerLocator.get();
} }
```

Listing 2: une même classe service

Approche qui va vers le découplage. SmtpServer peut être implémentée sous la forme d'une abstraction ou d'une interface. EmailService peut ignorer les paramètres permettant la construction de smtpServer. La dépendance est bien toujours présente, et les tests sont plus difficiles (indirection via SmtpServerLocator)

Pattern ServiceLocator

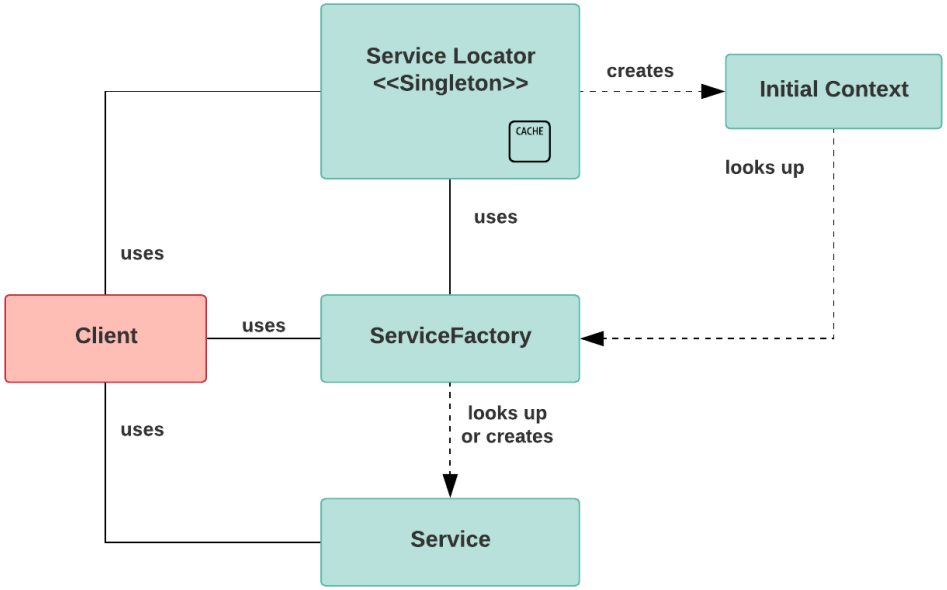

Figure: Modèle conceptuel ServiceLocator

La création de la dépendance est toujours sous la responsabilité de l'objet client (l'objet va chercher ses dépendances et ServiceLocator $\sim$ anti-pattern)

Crédit image : https://springframework.guru/service-locator-pattern-in-spring/

Solution avec injection de dépendance

```
public class EmailService implements Emailing
{ private SmtpServer smtpServer;
public EmailService(SmtpServer smtpServer) {
this.smtpServer = smtpServer;}

public void sendEmail(String to, String subject, String body)
{ // Code mobilisant la variable d'instance smtpServer } }
```

Listing 3: DI au niveau du constructeur

Solution qui garde les avantages du service locator , tout en rendant le lien de dépendance explicite entre EmailService et SmtpServer : une dépendance est injectée soit par le constructeur, soit par le setter, soit via une interface

Toujours injection de dépendance au niveau de l'accesseur

```
public class EmailService implements Emailing
{ private SmtpServer smtpServer;
public void setSmtpServer(SmtpServer smtpServer) {
 this.smtpServer = smtpServer;  }

public void sendEmail(String to, String subject, String body)
{ // Code mobilisant la variable d'instance smtpServer } }
```

Listing 4: DI au niveau accesseur

Il s'agit d'une partie de la solution puisqu'une partie du programme doit créer une instance de SmtpServer et une instance de EmailService en passant l'objet SmtpServer en paramètre. Spring va réaliser ces opérations au moyen d'un conteneur IoC

DI avec Spring et usage des annotations

```
@Service
public class EmailService implements Emailing {
private SmtpServer smtpServer;
@Autowired
public EmailService(SmtpServer smtpServer) {
this.smtpServer = smtpServer;}
public void sendEmail(String to, String subject, String body) {
// Code to send email using the smtpServer instance variable
}}
```

Listing 5: DI au niveau accesseur

Le conteneur/composant IoC gère le cycle de vie de tout un ensemble d'objets (et de leurs dépendances) et est constitué d'un ou de plusieurs contextes d'application. Au lieu de créer des dépendances, on les injecte depuis le framework. L'annotation @Autowired (auto-cablage) indique à Spring d'injecter automatiquement la dépendance à l'objet SmtpServer quand il crée l'instance de la classe EmailService

Retour sur les points d'importance avec Spring

- séparation des préoccupations
- basé composants (forte cohésion et faible couplage)
- mise en œuvre de la modularité via des patrons de conception
  - ServiceLocator (mais aussi Singleton, Factory)
  - Dependency Injection
- comportement d'un composant défini en terme d'interfaces fournies et requises

Rappel de ce qu'est un composant

- Un composant est une unité logicielle qui :
  - décrit de façon explicite ses :
    1. fonctionnalités fournies : signatures des opérations qu'il offre (comme dans les modules)
    2. fonctionnalités requises : signatures des opérations dont il a besoin pour implémenter les fonctionnalités fournies
    3. son architecture interne : liste des instances de ses composants internes et leurs interconnexions
  - est sujet à une instanciation et à une connexion avec d'autres composants (qui ont besoin de ses opérations / qui répondent à ses besoins -fournissent ses opérations requises-)

Crédit slide : Chouki Tibermacine `https://gitlab.com/ctiber/java-web`

UNIVERSITÉ DE MONTPELLIER

Modularité dans les composants logiciels

- Interfaces requises explicites :
  - Un composant ne dépend pas directement d'un autre composant
  - Il requiert un certain nombre d'opérations (spécifiées dans une ou plusieurs interface(s) : type abstrait) que peut fournir (implémenter) un autre composant
- Dépendances entre composants mises en place par un architecte :
  - Un rôle différent de celui qui développe les composants (qui sont à connecter, et destinés pour la réutilisation) : développeur par la réutilisation

Crédit slide : Chouki Tibermacine https://gitlab.com/ctiber/java-web

Composant EmailService

```
public class EmailService implements Emailing {
public ISmtpServer smtpServer;
public IAddressBook addressBook;

public EmailService(ISmtpServer smtpServer) {
this.smtpServer = smtpServer;}

public void sendEmail(String subject, String body) {
String address = addressBook.getAddress();
}}
```

Listing 6: Avec interfaces

Emailing est l'interface fournie ; ISmtpServer et IAddressBook sont les interfaces requises

Illustration EmailService

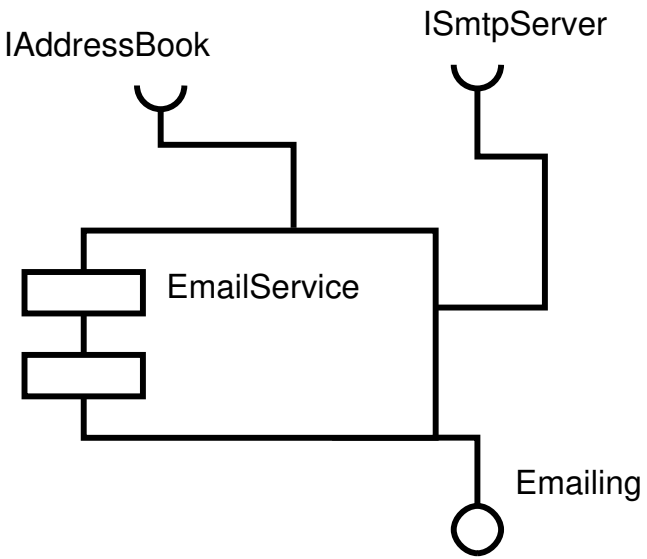

Figure: Composants et interfaces

Inversion de contrôle (IoC) et faible couplage

- au lieu de gérer manuellement les dépendances entre objets au sein de l'application, déléguer l'instanciation et la liaison des objets au framework (ici Spring)
- via une entité externe à l'application (le framework est dit injecteur)
  - création des instances
  - définition des liens entre instances

UNIVERSITÉ DE MONTPELLIER

Frameworks d'injection automatique de dépendances

- L'injection de dépendances peut être réalisée de façon automatique
- Il existe des frameworks Java proposant des solutions d'injection automatique de dépendances
- Quelques frameworks gratuits :
  - Spring : (le framework qui a rendu la DI populaire)
  - Guice de Google (le plus simple)
  - PicoContainer (l'un des premiers frameworks)
  - Des projets Apache : Avalon et HiveMind
  - Chez Oracle : DI dans JEE (inspirée de Spring et Guice)

Crédit slide : Chouki Tibermacine https://gitlab.com/ctiber/java-web

UNIVERSITÉ DE MONTPELLIER

IoC à l'échelle du temps

Genèse et ligne de vie

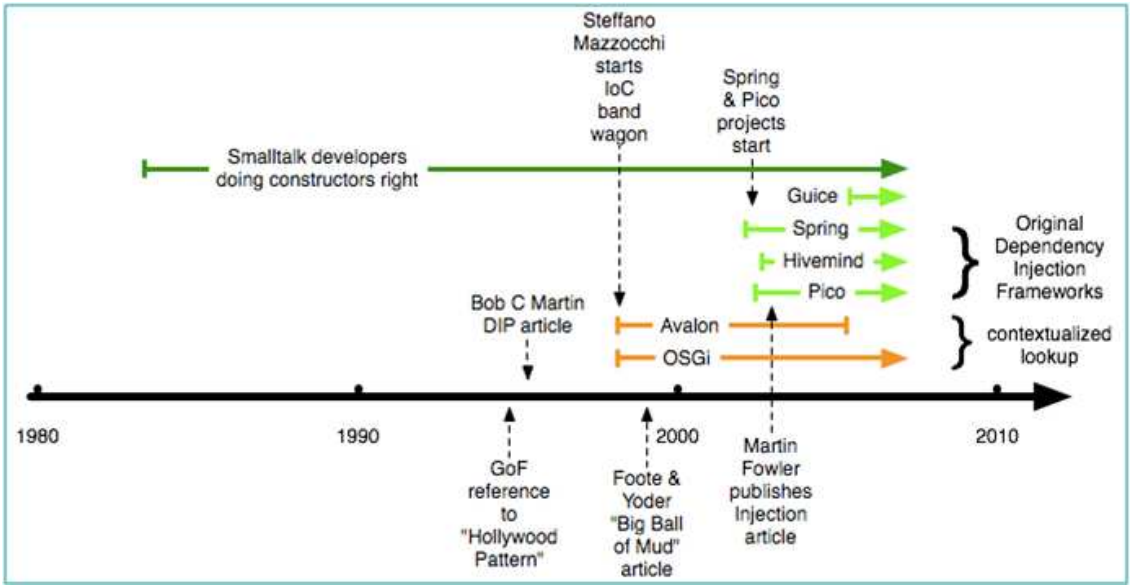

Figure: Timeline : injection de dépendances

voir `http://igm.univ-mlv.fr/~dr/XPOSE2010/guicespring/di_presentation.html`

Injection de dépendances (DI)

Une des formes possibles d'IoC, DI un patron de conception pour découpler les dépendances en POO

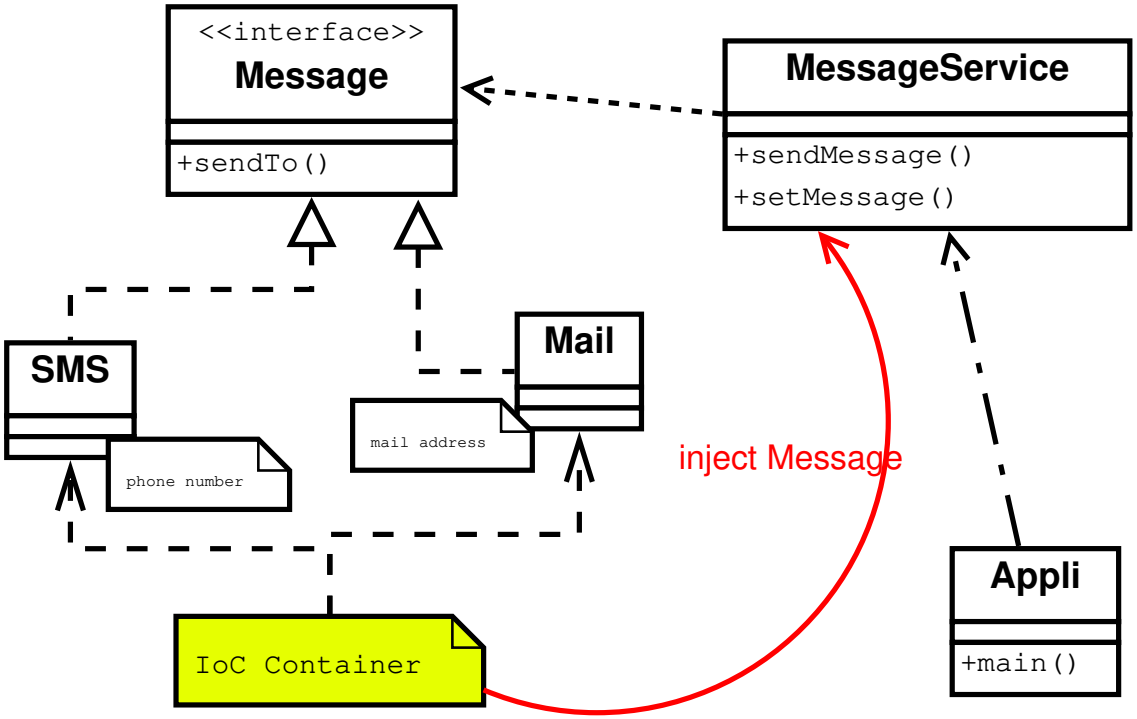

Figure: Principe général DI

Les dépendances sont définies dynamiquement à l'exécution

Auto-connecter des beans grâce aux annotations

- Activer les annotations pour l'auto-connexion :
  - Les annotations Spring : @Autowired, @Component, @Service, @Repository ...
  - Les annotations standards de Java EE > 6 : (implémentées dans Spring) : @Inject, @Qualifier, ...

UNIVERSITÉ DE MONTPELLIER

Auto-connecter un bean avec @Autowired

- Appliquer l'annotation à un setter :

```
@Autowired
public void setEditor(Editor e) {this.editor = e;}
```

Ici, l'auto-connexion se fera par type

- L'appliquer sur n'importe quelle méthode :

```
@Autowired
public void voiciUnEditor(Editor e) {this.editor = e;}
```

- L'appliquer sur une propriété directement :

```
@Autowired
private Editor editor;
```

- Si aucun ou au contraire plusieurs beans peuvent être utilisés dans l'auto-connexion, une exception est levée

Définir des auto-connexions optionnelles

- L'annotation @Autowired impose un contrat fort : auto-connexion obligatoire
- Possibilité d'avoir une exception : NoSuchBeanDefinitionException (c'est mieux qu'un NullPointerException plus loin dans l'exécution !)
- Définir une interface requise optionnelle d'un composant :

```
@Autowired(required=false)
private Editor editor;
```

Si aucun editor n'est trouvé, cette propriété reste non renseignée (Avec un peu de chance, on a prévu dans le code l'édition de texte sans mention à l'éditeur : if(editor != null) ...)

Crédit slide : Chouki Tibermacine https://gitlab.com/ctiber/java-web

Les composants du niveau Service

contient l'ensemble de la logique métier de l'application (matérialisable à l'aide de cas d'usage)

- séparation des préoccupations : logique métier séparée de ce qui relève des requêtes/réponses HTTP.
- réutilisabilité et maintenabilité : les services encapsulent la logique métier et peuvent être appelés par différents contrôleurs
- support aux transactions : périmètre transactionnel délimité au niveau de la couche. Validation ou annulation pour maintenir l'intégrité des données
- testabilité : les services peuvent être testés de manière isolée des autres couches. Tests unitaires sur la base de scénarios sans faire appel aux requêtes HTTP ou à la base de données
- encapsulation de la complexité de la logique métier (lisibilité, flexibilité et scalabilité) par le biais de fonctionnalités faciles à ajouter

Place des Services au sein de l'architecture

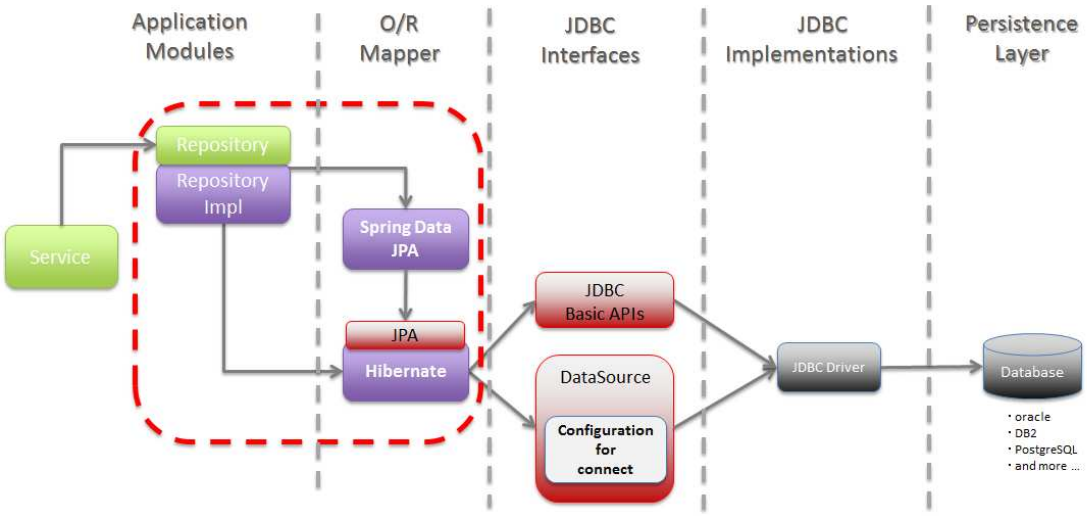

Figure: Le niveau métier mobilisant le niveau domaine

Crédit image `terasoluna.org`

Use Case TP : Service autour du Campus

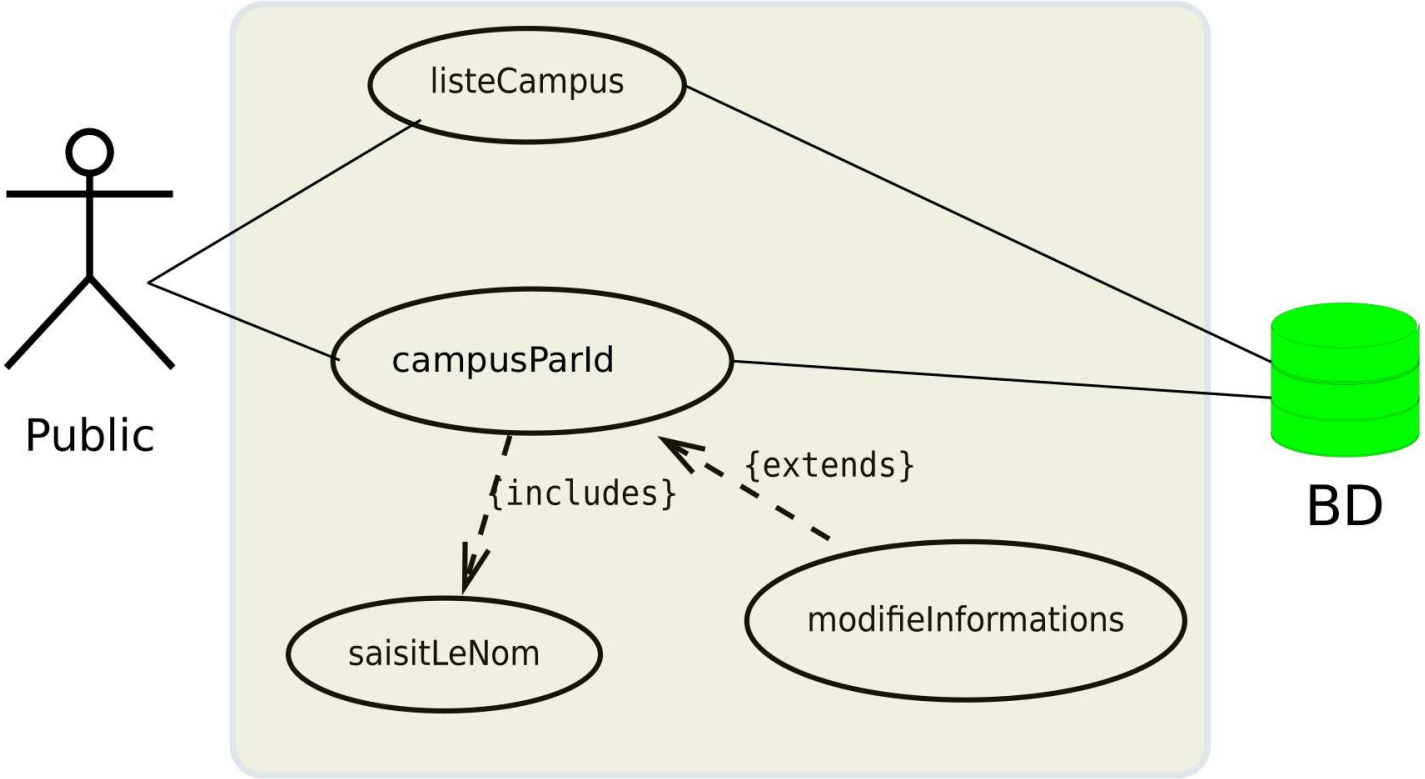

Figure: Exemple des tâches dévolues au service CampusService

Code interface ICampusService

```
package Ex.service;
import java.util.List;
import java.util.Optional;

import Ex.modele.Campus;

public interface ICampusService {
  Campus createCampus(Campus campus) ;
  Campus modifieInformations(String nomC, Campus updatedCampus) ;
  void deleteCampus(String id);
  List<Campus> listeCampus();
  Optional<Campus> campusParId(String nomC);
  void campusParVille(String ville) ;
}
```

Code classe CampusService (1)

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import jakarta.transaction.Transactional;
import java.util.Optional;
import java.util.stream.StreamSupport;
@Service
public class CampusService implements ICampusService {

    @Autowired
    private CampusRepository cr;

     public List<Campus> listeCampus() {
         return cr.findAll();     }

     public Optional<Campus> campusParId(String nomC) {
         return cr.findById(nomC);
     }
```

Exemple de code pour CampusService (2)

```
 public void campusParVille(String ville)
 {  Iterable<Campus> e = cr.findByVille(ville);
 System.out.println("La liste des campus de "+ville);
 if (StreamSupport.stream(e.spliterator(), false).count() != 0)
 {
  for (Campus c : e)
      { System.out.println(c.toString());}
    } else System.out.println("pas de campus connu sur "+ville);
}

@Transactional
public Campus createCampus(Campus campus) {
    return cr.save(campus);
 }
```

Exemple de code pour CampusService (3)

```
    @Transactional
  public Campus modifieInformations(String nomC,
          Campus updatedCampus) {
  return cr.findById(nomC)
        .map(campus -> {
        campus.setVille(updatedCampus.getVille());
        campus.setBatiments(updatedCampus.getBatiments());
        return cr.save(campus);
        })
        .orElseThrow(()
        -> new EntityNotFoundException("Not found: " + nomC));
  }
}
```

Rappel CampusRepository

```
import java.util.*;
import org.springframework.data.jpa.repository.*;
import org.springframework.stereotype.Repository;
import Ex.modele.Campus;

@Repository
public interface CampusRepository extends
        JpaRepository<Campus, String> {

  List<Campus> findByVille(String ville);

  @Query("SELECT c.nomC, bts.codeB, sls.numS FROM Campus c "
      + "JOIN c.batiments bts join bts.salles sls")
  List<String[]> campusEtCodeBEtNumS();
}
```

Illustration CampusService

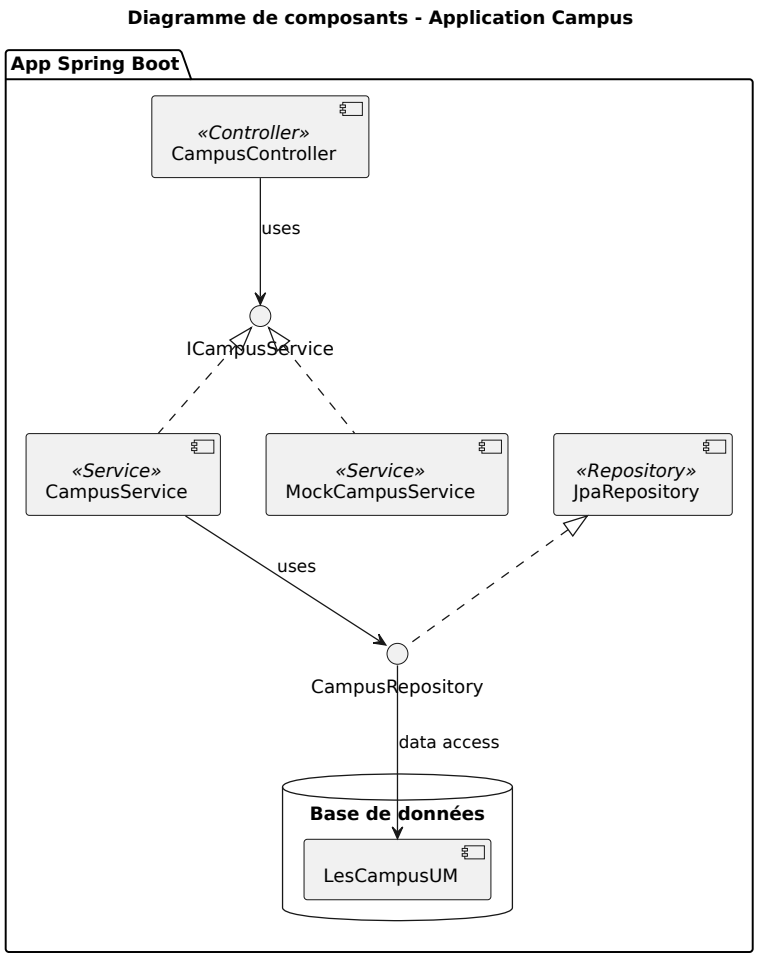

Figure: Service et Repository pour les campus

Un premier contrôleur

```
package Ex.control;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.*;

import Ex.service.CampusService;

@Controller
public class CampusController {

  @Autowired
  private ICampusService cs;

  @GetMapping("/listeCampus")
  public String home(Model model) {
    model.addAttribute("campuses", cs.listeCampus());
    return "campuses";
  }

}
```

Application Springboot

```
package Ex;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class DemoApplication {

  public static void main(String[] args) {
    SpringApplication.run(DemoApplication.class, args);
  }
```

En vue de tests : MockCampusService

Une interface est une spécification de comportement (ou contrat). Ici 2 implémentations pour ICampusService :

1. CampusService
2. MockCampusService

Le conteneur voit 2 classes et ne peut en injecter qu'une : erreur

```
@Primary (implementation par defaut)

@Service
@Primary
public class MockCampusService implements ICampusService { ...}
```

En vue de tests : MockCampusService

```
package Ex.service;
import org.springframework.context.annotation.Primary;
...
import jakarta.annotation.PostConstruct;
import java.util.*;
import java.util.concurrent.ConcurrentHashMap;
@Service
@Primary
public class MockCampusService implements ICampusService {

private final Map<String, Campus> storage = new ConcurrentHashMap<>(
    @PostConstruct
    public void init() {
      storage.put("Triolet", new Campus("Triolet", "Montpellier"));
      storage.put("Balard", new Campus("Balard", "Montpellier"));
    }

    @Override
    public List<Campus> findAll() {
        return new ArrayList<>(storage.values());
    } .....
    }
```